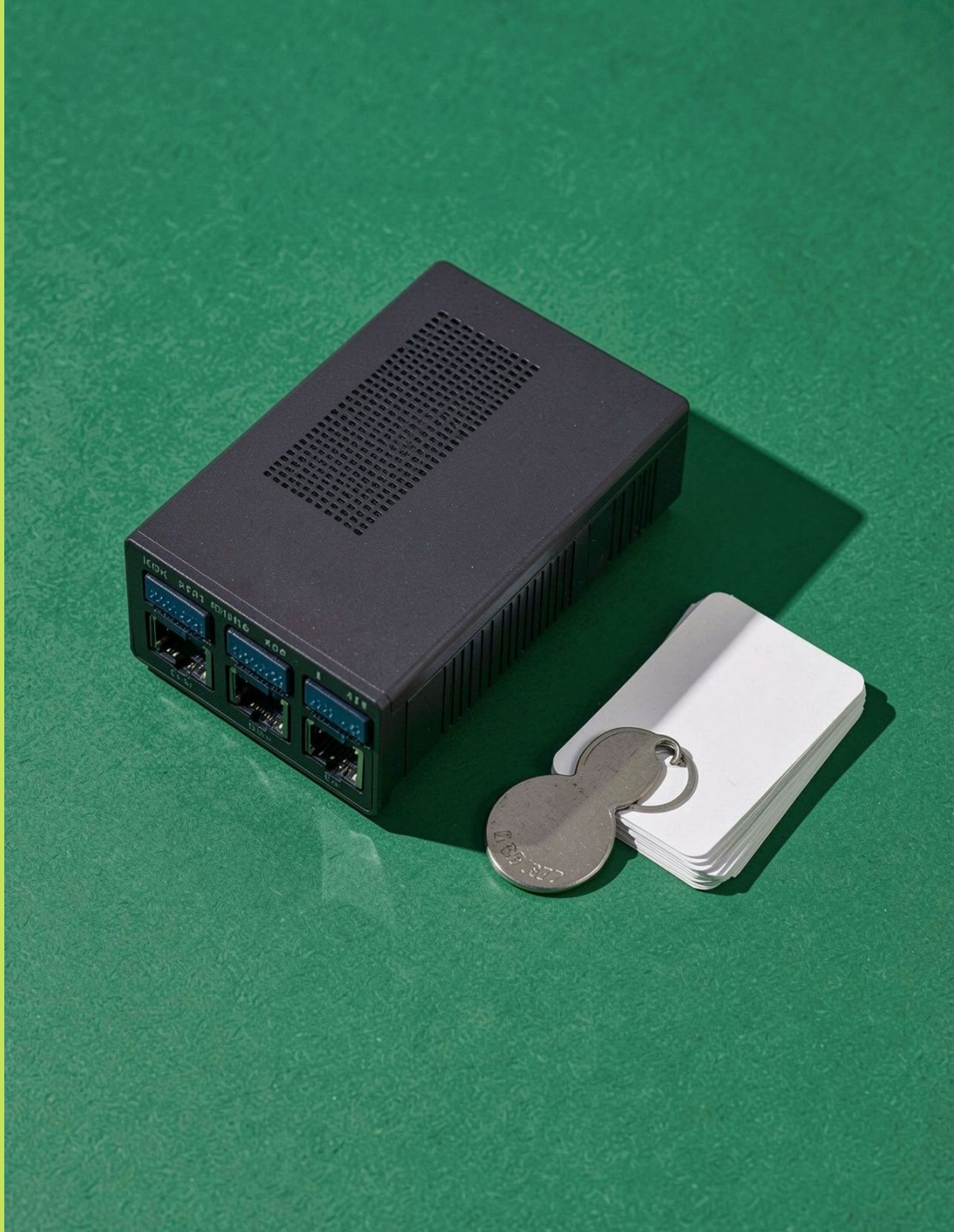
SQL → NoSQL

OptiQuery

Praveen Kumar – 24BKT0043
Shanti Swaroop – 24BKT0046

Custom SQL-to-NoSQL Query Compiler and Cost-Based Optimizer



Why this project is needed

1. NoSQL databases use different query languages and APIs.

2. Developers must learn platform-specific syntax.

3. Simple translation can create slow and wasteful queries.

The problem with current translation tools

DIRECT MAPPING

Many tools map SQL directly to NoSQL syntax.

MISSED REWRITES

They often miss filter reordering and projection pushdown.

EXTRA WORK

Nested or filtered queries may fetch more data than necessary.

What OptiQuery aims to do



Translate

Translate a useful subset of SQL into NoSQL execution code.



Compile

Use compiler phases instead of direct string conversion.



Choose

Choose better execution plans with a lightweight cost model.

Objectives

The primary objective of this project is to construct a comprehensive compiler pipeline consisting of multiple integrated phases aligned with compiler design principles.

These include lexing and parsing standard SQL queries into an Abstract Syntax Tree (AST) using context-free grammars (Modules 1 and 2), constructing robust symbol tables, and conducting semantic analysis and type checking via Syntax Directed Definitions (Module 3).

Further objectives involve generating intermediate evaluation trees (Module 4) and implementing code optimization passes using a Directed Acyclic Graph (DAG) representation of basic blocks to perform filter reordering, common subexpression elimination, and projection pushdowns (Module 5).

Additionally, the project aims to design a lightweight cost model that factors in document size estimation and index availability, implement peephole and loop-restructuring optimizations for query execution flows, and target multiple NoSQL backends like MongoDB aggregation pipelines and Redis command dialects.

Literature Survey

Classic database literature, such as System R and the Volcano/Cascades frameworks, establishes the foundation of cost-based optimization via relational algebra equivalence rules, though traditionally restricted to relational engines.

Recent academic and industrial frameworks—such as Couchbase's SQL++ query language and hybrid polyglot engines like SQL Server's PolyBase—have explored bridging declarative queries to semi-structured and document stores.

Meanwhile, document and key-value stores utilize custom JSON-based querying and aggregation APIs that lack built-in declarative optimization layers, placing the optimization burden squarely on application developers.

Furthermore, code optimization techniques using DAGs for basic blocks, loop restructuring, and algebraic simplifications are well-documented in traditional compiler design (such as LLVM or GCC), but research on adapting these compiler IR transformations specifically for semi-structured data traversal and NoSQL latency models remains an evolving domain.

Research Gap

Most existing database translation tools, middleware, or ORMs perform static, rule-free mapping from SQL strings to NoSQL queries without considering dynamic data distribution costs, document schema nesting, or algebraic equivalence.

There is a noticeable lack of lightweight compilers that treat NoSQL query generation as a formal compiler pipeline—specifically utilizing DAG-based basic block restructuring, intermediate code generation, and cost-based query rewrites to systematically minimize data traversal costs before targeting NoSQL stores.

The compiler pipeline

Lex and parse the SQL query → **Check meaning** types and valid references → **Build** an intermediate evaluation tree → **Optimize** the plan → **Generate** MongoDB or Redis code

From SQL to an intermediate plan

SQL is converted into an Abstract Syntax Tree. Semantic analysis checks types and names. An intermediate representation makes later rewrites possible. The plan stays separate from the final database syntax.



How the optimizer improves queries

01

Reorder filters

So selective conditions run earlier.

02

Remove repeated calculations

With common subexpression elimination.

03

Push projections

Closer to the data source.

04

Use a DAG

To share repeated expressions.

Choosing a lower-cost execution plan

Lowest Estimated Plan Cost

Equivalent plan comparison

Compare alternatives before codegen

Filtering cost

Cost of predicate evaluation

Traversal cost

Index and scan expenses

Data movement cost

Shuffling and I/O overhead

Estimate the work needed by different plans.

Consider filtering, data movement, and traversal cost.

Compare equivalent plans before code generation.

Select the plan with the lowest estimated cost.

Expected result

PIPELINE

A formal compiler pipeline for SQL-to-NoSQL translation.

EFFICIENCY

Less redundant data fetching and simpler target code.

TARGETS

Support for MongoDB aggregation pipelines and Redis commands.

BRIDGE

A practical bridge between compiler theory and NoSQL systems.